PHOENIXTEKK

Workplace Marketplace · Intune Solutions

Active Directory Capacity & Architecture Assessment — Project Group-Provisioning (RID) Review

Implementation Solution Guide

Version 1.0.0 · 2026-07-03

workplace.phoenixtekk.com

Contents

1. Executive Summary . . . 3
2. Restated Requirement (what is actually being asked) . . . 3
3. Current State & Workload . . . 3
4. RID Fundamentals (why the prior warning happened, and why it's low-risk now) . . . 4
5. RID Consumption Projection . . . 4
6. The Risks That *Do* Matter (non-RID) . . . 5
7. Authorization-Architecture Review (is 3 groups/project the right model?) . . . 6
8. Recommendation & Conditions of Approval . . . 7
9. Monitoring — PowerShell . . . 7
10. Answers to the Specific Questions . . . 8
11. Decision . . . 9
Appendix A — PowerShell Scripts 10

1. Executive Summary

HDR is standing up an automated provisioning system that creates **three Active Directory security groups per engineering project** — a **Read**, a **Write**, and a **Privileged Write** group — at a projected rate of **30–40 projects per week**. A prior generation of this tooling created 5 groups + a shared mailbox per project and was scaled back after Microsoft raised a concern about the *rate* of security-principal creation and RID consumption.

Bottom line: RID exhaustion is not a realistic risk for this workload. The domain has consumed **1,478,144 of 1,073,741,823 RIDs (~0.138%)**. The proposed application adds at most **~6,240 groups per year**; against the **1,072,263,679 remaining** RIDs, this workload alone would take on the order of **170,000+ years** to exhaust, and even the *entire* organization's blended object-creation rate leaves a runway measured in **thousands of years** — with a documented **safety valve** (the extended RID pool) that doubles capacity if ever needed.

However, the assessment should not be approved on RID availability alone. The real, near-term risks of a per-project 3-group model are **Kerberos token bloat** (for users/service accounts attached to many projects), **group sprawl and lifecycle debt** (abandoned project groups), and **operational/administrative overhead** — none of which are RID problems. This document approves the model **with governance guardrails** and provides the monitoring and lifecycle controls required to keep it sustainable.

Recommendation: APPROVE the 3-group-per-project model, conditional on the guardrails in §8.

2. Restated Requirement (what is actually being asked)

> *Determine whether the proposed per-project Active Directory group-provisioning model is **sustainable over the long term** — without introducing RID exhaustion, directory scalability issues, or operational risk — and provide recommendations for a scalable design aligned with Microsoft Active Directory best practices.*

The question "can we create three groups?" is the implementation detail. The decision this assessment supports is an **architectural go/no-go with conditions**.

3. Current State & Workload

Item	Value
Groups per project	3 (Read, Write, Privileged Write)
Projects per week	30–40
Groups per week	90 – 120
Groups per year	**4,680 – 6,240** (52 weeks)
Prior model (retired)	5 groups + 1 shared mailbox/project
RIDs consumed (reported)	**1,478,144**
RID maximum (default 30-bit pool)	**1,073,741,823** ($2^{30} - 1$)
Percent consumed	**~0.138%**

Item	Value
RIDs remaining	**1,072,263,679**

*(Note: the maximum-workload annual figure is 40 × 3 × 52 = **6,240 groups/year**, not ~1,500 — the higher number is used for all worst-case projections below.)*

4. RID Fundamentals (why the prior warning happened, and why it's low-risk now)

• Every **security principal** (user, computer, group) is assigned a unique **SID** = *domain SID* +

RID. RIDs come from a **per-domain pool** managed by the **RID Master** FSMO role holder.

• The RID Master hands each DC blocks of **500 RIDs** at a time; a DC requests a new block when it drops

below ~50% of its current block.

• The default pool is **30-bit** → **2³■ – 1 = 1,073,741,823** RIDs per domain.

• **RIDs are never reclaimed.** Deleting a group does **not** return its RID to the pool. So consumption

is **cumulative over the life of the domain** — this is why *rate* matters and why the prior 5-groups+ mailbox model drew a caution: it was the trajectory, not an immediate cliff.

• **Safety valve:** on Windows Server 2012+, the **extended RID pool** can be unlocked to raise the pool

ceiling to **2^{31} = 2,147,483,648** (doubling capacity). This is a one-way, supported operation held in reserve — it is *not* needed here, but it exists.

5. RID Consumption Projection

This application's contribution only:

Metric	Value
Worst-case groups/year	6,240
RIDs remaining	1,072,263,679
Years to exhaust (this app alone, worst case)	**~171,800 years**
Years to exhaust (this app alone, low case 4,680/yr)	~229,100 years

Whole-organization blended view (RIDs are consumed by *all* principals ever created — users, computers, and every other group source — not just this app):

Assumed total org RID burn / year	+ this app (6,240)	Years to exhaust remaining
50,000	56,240	~19,070 years
100,000	106,240	~10,090 years

Assumed total org RID burn / year	+ this app (6,240)	Years to exhaust remaining
250,000	256,240	~4,185 years
500,000	506,240	~2,118 years

Even under aggressive assumptions, the runway is **millennia**. The incremental **6,240 RIDs/year** from this application is **statistically negligible** against both the remaining pool and the extended-pool safety valve.

Finding: RID exhaustion risk = **Very Low**. Not a basis to block the design.

6. The Risks That *Do* Matter (non-RID)

RID exhaustion is the wrong thing to worry about at this scale. The following are the real, design-relevant impacts of proliferating per-project groups.

*6.1 Kerberos Token Bloat *(highest practical risk)**

• A user's Kerberos ticket (PAC) carries the **SID of every group they belong to** (plus SID history and, under AGDLP, the domain-local resource-group SIDs).

• Default **MaxTokenSize = 48,000 bytes** (Server 2012+). There is also a hard ceiling of roughly **1,015 group SIDs** before authentication fails outright.

• **Where this bites:** a user, **service account, or automation identity attached to many projects**. At 3 groups/project, a person on **100 projects consumes ~300 project-group SIDs** — before counting their baseline enterprise groups. Power users, PMs, and service accounts that span the portfolio are the exposure, not the average engineer.

• **Mitigation:** monitor token size for high-membership principals; avoid deep nesting; scope service accounts to only the projects they truly need; consider role-resource separation (§7).

*6.2 Group Sprawl & Lifecycle Debt *(highest operational risk)**

• 6,240 groups/year with **no deprovisioning** = **62,400 groups in 10 years** from this app alone.

• Without automated cleanup, **completed/abandoned projects leave orphaned groups** that inflate backups, replication, search, and audit scope, and erode least-privilege over time.

• **Mitigation:** lifecycle automation tied to project state (see §8) is the single most important guardrail.

6.3 Replication & Global Catalog

• Security groups and their memberships replicate to all DCs; universal groups and GC-replicated attributes add inter-site traffic. At thousands/year this is **modest but non-zero**; large-membership groups matter more than group *count* (use **linked-value replication** — Server 2003+ functional level — so membership changes replicate per-member, not whole-list).

6.4 Administration & Governance Overhead

• Naming collisions, ownership ambiguity, unclear scope, and audit fatigue scale with object count.

Requires **naming standards, a dedicated OU subtree, and ownership metadata**.

*6.5 SYSVOL — *not impacted**

• These are **permission/security groups**, not GPO objects. SYSVOL/GPO scale is **not** a factor unless the groups are used for GPO security filtering (they are not, per the described design).

6.6 Backup / DIT size

• Each object adds to the **NTDS.DIT** and system-state backup. Thousands/year is small relative to typical enterprise directories, but compounds with §6.2 if cleanup is absent.

7. Authorization-Architecture Review (is 3 groups/project the right model?)

The 3-group model (**Read / Write / Privileged Write**) is a **legitimate least-privilege design** and is worth keeping — provided it follows Microsoft's canonical pattern and has lifecycle governance.

7.1 Apply AGDLP (or AGDLP-per-resource), don't flatten

• **Accounts → Global groups (roles) → Domain Local groups (resource permissions) → Permissions.**

• Best-practice mapping: the **per-project Read/Write/Privileged groups are the Domain Local "resource"

groups **applied to the project's file shares/SharePoint/resources.** Users are added via Global "role" groups**, which are nested into the resource groups.

• This keeps ACLs clean, supports multi-domain/forest, and localizes churn to membership (linked-value

replication) rather than ACL rewrites.

7.2 Options to reduce object creation (if desired — not required)

Approach	Effect	Trade-off
Keep 3 resource groups/project (recommended)	Clean least-privilege; RID-safe	Highest object count (acceptable here)
1 project group + role via nested global groups	Fewer per-project objects (what the prior app did)	Permissions logic moves into nesting/RBAC; less transparent ACLs
RBAC via enterprise role groups + resource groups	Reuse existing roles; fewer new per-project objects	Requires a maintained role catalog
Entra ID groups / dynamic groups	Good for **cloud** resources	**Does not** set on-prem NTFS/SharePoint-Server ACLs; not a substitute for AD groups on file resources

Recommendation: retain **3 domain-local resource groups per project** (the model is fine and is the clearest least-privilege expression), implemented through **AGDLP**, with the governance in §8. Only collapse to fewer groups if token bloat (§6.1) becomes a measured problem for portfolio-spanning identities.

8. Recommendation & Conditions of Approval

APPROVE the 3-group-per-project provisioning model. RID capacity is a non-issue. Approval is **conditional on the following guardrails**, which address the risks that actually matter:

1. **Lifecycle automation (mandatory).** Tie group lifetime to project state: on project

close/archive, **disable** → **stage** → **delete** the three groups on a defined timer (e.g. 30/60/90 days). This is the single most important control — it converts an ever-growing pile into a steady state.

1. **Dedicated OU subtree + naming standard.** e.g.

`OU=ProjectGroups,OU=Provisioned,DC=…` and `PRJ-<ProjectID>-<READ|WRITE|PRIV>` — deterministic, collision-free, auditable, easy to target for cleanup and reporting.

1. **AGDLP implementation.** Project groups are **Domain Local** resource groups; membership comes via

Global role groups (§7.1).

1. **Ownership metadata.** Stamp each group with owner, project ID, and creation date (`info`/`managedBy`/

extension attributes) so orphan detection and recertification are automatable.

1. **Monitoring (mandatory):**

• **RID pool** — trend consumption and RID Master health (script in §9). Alert at ≥ **80%** of the

30-bit pool (we are at 0.138%).

• **Token size** — watch high-membership users/service accounts against MaxTokenSize (§6.1).

• **Orphan/growth report** — weekly count of project groups vs. active projects.

1. **Access recertification.** Periodic review of Privileged Write membership (least privilege drifts).
2. **Cap portfolio-spanning identities.** Ensure service/automation accounts are scoped to needed

projects only, to bound token growth.

With these controls, the model is **sustainable for the foreseeable life of the domain**.

9. Monitoring — PowerShell

9.1 RID pool consumption & RID Master health

```
# Remaining RIDs, % consumed, and RID Master identity
$domain      = Get-ADDomain
$ridMaster   = $domain.RIDMaster
$rootDSE     = Get-ADRootDSE
$ridSet      = Get-ADObject "CN=RID Manager$,CN=System,$($domain.DistinguishedName)" -Property rIDAvailabl
```

```
$pool        = $ridSet.rIDAvailablePool
$high        = [math]::Floor($pool / ([math]::Pow(2,32)))   # pool ceiling (max RIDs)
$low         = $pool -band ([math]::Pow(2,32) - 1)          # next RID to be issued (consumed)
$consumed    = $low
$max         = $high
$remaining   = $max - $consumed
$pct         = [math]::Round(($consumed / $max) * 100, 4)

[pscustomobject]@{
    RIDMaster       = $ridMaster
    RIDsConsumed    = $consumed
    RIDMaximum      = $max
    RIDsRemaining   = $remaining
    PercentConsumed = "$pct %"
} | Format-List
```

9.2 Project-group growth & orphan report

```
# Count provisioned project groups and flag ones whose project is closed.
$ou = "OU=ProjectGroups,OU=Provisioned,DC=example,DC=com"
$groups = Get-ADGroup -SearchBase $ou -Filter * -Property whenCreated, info, managedBy
"Total project groups: $($groups.Count)"
$groups | Group-Object { ($_.Name -split '-')[1] } |   # by ProjectID
    Select-Object Name, Count | Sort-Object Count -Descending | Select-Object -First 20
# Orphans = groups whose ProjectID is not in the active-projects list you supply
```

9.3 Token-size exposure for high-membership principals

```
# Approximate token contributors: count group memberships (incl. nesting) per user of interest.
$user = Get-ADUser "jdoe" -Property memberOf
$tokenGroups = (Get-ADUser $user.DistinguishedName -Properties tokenGroups).tokenGroups
"SID count in token (approx): $($tokenGroups.Count)"   # watch against ~1,015 hard cap / MaxTokenSize
```

*(Enable the **RID pool alert** in your monitoring platform at 80% of `RIDMaximum`; today it would read 0.138%.)*

10. Answers to the Specific Questions

#	Question	Answer
1	Will ~1,500 (really ~6,240) groups/year become a scalability problem?	**No** for RIDs; **manageable** for operations **with lifecycle cleanup** (§8).
2	Is RID consumption a realistic concern?	**No.** 0.138% consumed; incremental burn is negligible; extended pool in reserve.
3	Projected RID pool lifetime?	**Thousands of years** even under aggressive whole-org assumptions (§5).
4	Better architectural approaches?	Keep 3 groups but implement **AGDLP + lifecycle + naming**; consider RBAC role reuse only to bound token bloat (§7).

#	Question	Answer
5	Other AD impacts?	**Token bloat**, **group sprawl/lifecycle**, replication/GC, backup, admin overhead — addressed by §8. SYSVOL not impacted.

11. Decision

> ***Approved to proceed*** *with the 3-group-per-project provisioning model,* ***conditional on implementing the §8 guardrails*** *(lifecycle automation, dedicated OU + naming standard, AGDLP, ownership metadata, and RID/token/orphan monitoring). RID capacity is* ***not*** *a limiting factor. Re-review if per-project group count increases, if portfolio-spanning identities approach token limits, or if RID consumption trend alerts fire at ≥ 80%.*

Reviewed by: Active Directory Architecture. Next review: on material design change or annually.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.